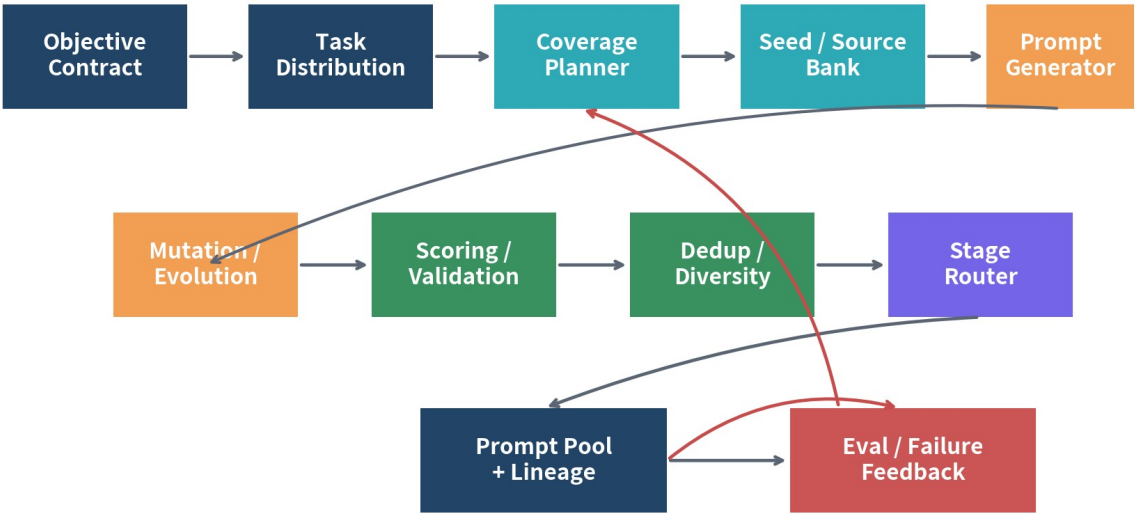


LLM 后训练之 Prompt Factory

深度研究与工程手册

从 Seed、Synthetic Prompt、Persona、Evolution、Real-user Mining 到 Failure-driven Data Flywheel

Prompt Factory：把“目标与任务分布”编译成可训练 Prompt 资产



研究日期：2026-08-13

执行摘要：Prompt Factory 到底是什么？

“Prompt Factory”不是学术界统一命名的单一算法，而是本文对现代后训练数据系统的一种工程抽象：它负责把 Objective Contract 与 Task Distribution 编译成可训练、可追踪、可验证、可迭代的 Prompt 资产。

核心定义 Prompt Factory = Coverage Planner + Source/Seed Bank + Generator + Mutation/Evolution + Validator + Diversity/Dedup + Stage Router + Lineage + Feedback Controller。

它的输出不是“几百万条问题”，而是一份带有能力标签、来源、难度、约束、真实性、可验证性、风险、去重簇、生成器、版本与使用阶段的 Prompt Pool。只有这样，Prompt 才能成为后训练可调度的训练经验，而不是不可解释的数据堆。

问题	错误理解	工程化理解
Prompt 从哪里来？	找一个大模型批量生成	组合真实用户、人类种子、模型生成、语料反推、persona、失败样本等来源
Prompt 越多越好吗？	数量越大越强	覆盖、复杂度、真实性、可验证性、去重后的边际训练价值更重要
Prompt 与 Response 一起生成？	一次生成完整 pair	Prompt 与 Completion 分层生产、分层验收；Prompt 合格不代表 Answer 合格
Prompt 是否跨 SFT/DPO/RL 复用？	一份池通吃	不同训练阶段需要不同 prompt 分布、可验证性与 on-policy 属性
Prompt Factory 何时结束？	数据集生成完就结束	Eval/生产失败持续回流，Prompt Pool 动态重加权 and 再生成

本文重点回答 know-why 与 know-how：为什么需要 Prompt Factory；不同生成范式解决什么缺口；如何设计 seed、复杂度、persona、真实用户分布；如何过滤、去重、避免 contamination；如何把 Prompt Factory 接到 SFT、DPO、RLVR、Agent 和失败驱动闭环。

六个核心结论

1. 先定义“经验空间”，再生成 Prompt：Objective 与 Task Distribution 应先于 Prompt Synthesis。
2. Prompt 的“有效任务率”与完整训练样本的“答案正确率”是两件事，必须解耦 Prompt Factory 和 Completion Factory。Self-Instruct 的人工抽检非常直观地证明了这一点。[S1]
3. 多样性不能只用词面去重来定义；至少要同时覆盖 intent、domain、persona、format、interaction、difficulty、context、language 与 tool/environment 维度。
4. 复杂度不是“变长”。Evol-Instruct/DEITA 更有价值的做法是加入约束、加深推理、具体化、组合多步要求，并验证复杂度提升是否仍然可解。[S3][S7]
5. 现代高质量配方越来越倾向 Hybrid Prompt Portfolio：真实用户 + 公开高质量数据 + skill-specific synthetic + persona + hard/failure prompts。Tulu 3 是一个公开的典型案例。[S12]
6. 真正先进的 Prompt Factory 是反馈控制器：Eval/线上失败 → Failure Taxonomy → Gap Queue → 定向生成/挖掘 → 过滤 → 新训练 → 再评测。

1. 边界：Prompt Factory 在后训练流水线中的位置

Prompt Factory：把“目标与任务分布”编译成可训练 Prompt 资产

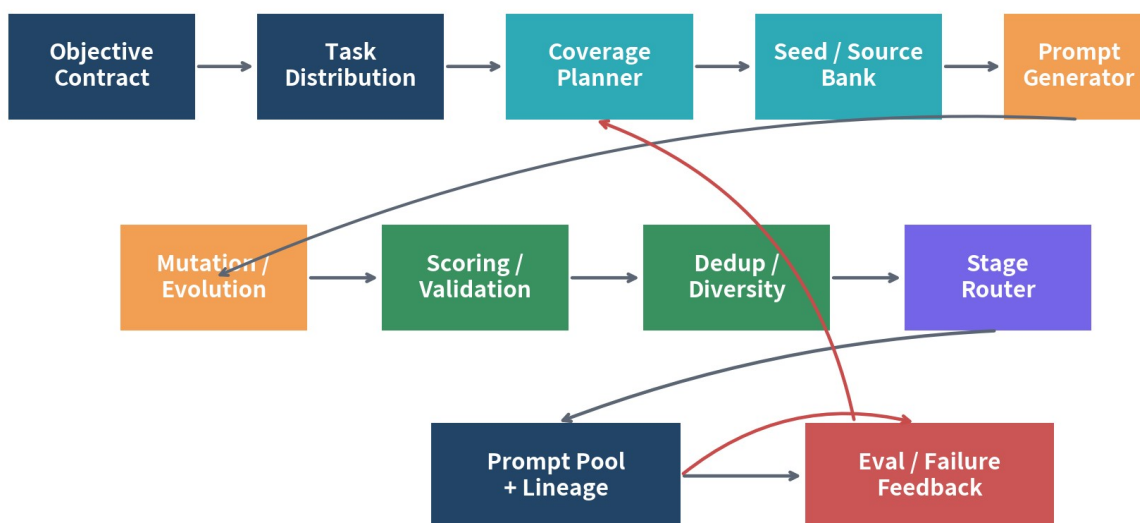


图 1：Prompt Factory 是 Objective/Task Distribution 与 Response/Rollout 之间的数据控制层。

上一模块 Task Distribution 决定“训练经验应该落在哪些区域、各自占多少预算”；Prompt Factory 决定“怎样生产足够多、足够真实、足够有区分度的问题实例去实现这个分布”。因此它不是简单的数据生成器，而是 Task Distribution 的执行器。

上游输入	Prompt Factory 责任	下游输出
Objective Contract	把抽象能力目标变成 Prompt 约束与验收条件	目标化 Prompt Pool
Task Distribution	把 $\text{capability} \times \text{difficulty} \times \text{domain}$ 等预算变成生成 quota	按 bucket 计数/权重的 Prompt
Eval Failure	把失败类型转成新的 gap 与 targeted prompt recipe	Hard/necessary prompts
Real-user logs	发现真实意图、语言、格式与异常分布	真实性增强 prompt
Corpus / Repo / Docs	构造 grounded、领域化、可验证问题	source-grounded prompt

关键接口契约 Prompt Factory 输出给 Completion Factory 的不是纯字符串，而应至少包含 prompt_id、task_bucket、source、difficulty、constraints、grounding、expected_verifier、lineage 与 contamination flags。

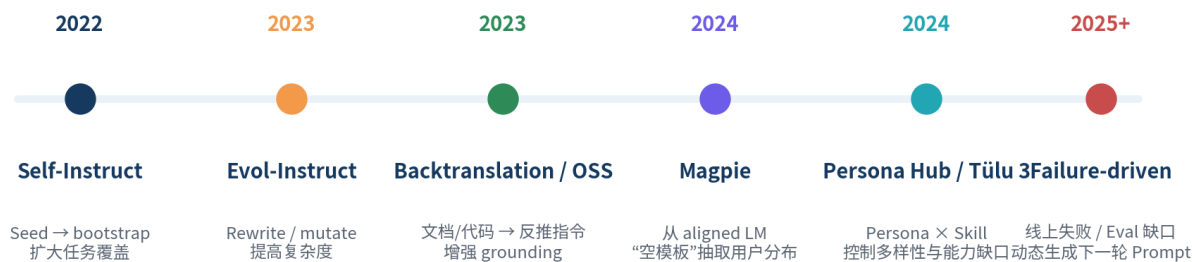
1.1 为什么不应该直接“让 GPT 生成 1M 个问题”？

- Generator distribution bias：一个模型会反复生成自己最熟悉、最常见的意图和表达方式。
- Easy-task collapse：若没有难度控制，生成内容往往集中在简单 QA、改写、摘要等高概率任务。
- Prompt-answer coupling：同一模型同时造问题和答案，会把生成器偏差写进两端。
- Coverage illusion：词面看起来不同，但语义意图高度重复。

- Contamination：若生成时显式或隐式接触评测样本，训练集可能泄漏 benchmark。
- No feedback control：一次性生成无法自动补齐模型下一轮真正的 capability gap。

2. Prompt Synthesis 的方法演进：每一代解决什么问题？

Prompt 生成范式的演进：从“扩量”走向“可控覆盖 + 真实分布 + 闭环补缺”



范式	核心输入	核心操作	主要解决	主要风险
Seed Bootstrapping	少量人工 seed	模型生成新 instruction + instance	低成本扩覆盖	复制 seed/模型偏见
Rephrase / Expansion	已有 instruction	改写、多样表达	表述多样性	语义仍同质
Instruction Evolution	简单指令	加约束/加推理/具体化	复杂度与 hard cases	不可解/伪复杂
Backtranslation	真实文本/文档	从输出反推用户指令	grounding / realism	受语料覆盖限制
Blank-template Elicitation	aligned chat template	让模型自然续写 user turn	减少 seed 限定	继承 aligned model 潜在分布
Persona-conditioned	persona × skill	从某类用户/角色角度生成	多样视角与长尾	persona 噱头化
Real-user Mining	真实交互日志	筛选/标签/匿名化	真实性	隐私、噪声、风险
Failure-driven	Eval/production failure	定向构造相似但不泄漏的任务	必要性 / gap closure	过拟合某套 eval

2.1 Self-Instruct : Prompt Factory 的“bootstrapping 原型”

Self-Instruct 从 175 个手写 seed task 起步，每轮抽取任务作为 in-context examples，让模型生成新 instruction，再判断任务类型、生成实例并用启发式规则过滤。它最终得到 52,445 条 instructions 和 82,439 个 instances。[S1]

最值得 Prompt Factory 工程借鉴的不是数量，而是“池化 + 回灌 + 过滤”的结构：seed pool → generate → filter → add to pool → repeat。Self-Instruct 还规定新 instruction 与已有 instruction 的 ROUGE-L 相似度需低于 0.7 才加入池中。[S1]

人工抽检项 (Self-Instruct)	Yes 比例	工程含义
-----------------------	--------	------

Instruction 描述有效任务	92%	“问题本身有效” 相对容易
Input 适合该 instruction	79%	输入生成开始出现明显噪声
Output 正确且可接受	58%	答案端是更困难的质量瓶颈
所有字段都有效	54%	Prompt 与 Completion 必须分层 QA

推论 Prompt Factory 最好只对“问题资产”负责到底；Response/Trajectory 应由独立 Completion Factory + verifier/judge 负责。否则一个看似 1M 样本的数据集，真正可训练信号可能远低于名义规模。

2.2 Unnatural Instructions : 极低人工成本的扩写

Unnatural Instructions 用 3 个 seed examples 提示模型生成第 4 个样本，先收集 64K examples，再通过 rephrase 扩展到约 240K instruction-input-output examples。[S2] 这说明 Prompt Factory 可以把“生成新任务”和“对同一任务增加自然语言表述多样性”拆成两类算子。

2.3 Evol-Instruct : 把“复杂度”变成显式可控变量

WizardLM 的 Evol-Instruct 从初始指令出发，逐步重写成更复杂的 instruction，再把不同复杂度的数据混合训练。[S3] 其核心价值不是让 prompt 变长，而是把 difficulty 作为工厂的可调旋钮。

- In-depth evolution：增加约束、加深问题、具体化、提高推理步骤。
- In-breadth evolution：改变任务方向，扩大 intent/skill 覆盖。
- 验收要求：复杂化后的任务必须仍然清晰、可解、非自相矛盾；否则只是“伪难题”。

3. 五类核心 Prompt 生成器：Know-Why 与 Know-How

3.1 Generator A : Seed-driven Bootstrap

设计项	Know-Why	Know-How
Seed 数量	太少容易锁定少数任务模式；太多提高人工成本	先覆盖 taxonomy 的主叶子节点，每个节点少量高质量 anchor
Seed 质量	生成器会放大 seed 的格式/价值观/歧义	写“规范 seed”，避免含糊、不可解、过多模板词
Seed sampling	固定 few-shot 会导致重复	分层随机抽样；混合人工 seed 与高分 synthetic seed
Pool update	只用原始 seed 难扩展	高分新任务可回灌，但限制 synthetic feedback 比例
Novelty gate	表面改写不等于新任务	词面 + embedding + intent/tag 多层去重

3.2 Generator B : Evolution / Mutation

可以把 prompt mutation 设计成“算子库”，而不是一个万能 rewrite prompt。每个算子负责改变一个可解释维度。

Mutation Operator	例子	适合能力	风险
Add constraint	限定格式、长度、必须满足多个条件	Instruction Following	约束冲突
Deepen reasoning	加入必须跨多步推导的条件	Math/Reasoning	伪装成难题
Concretize	加入具体实体、数据、边界条件	Realism	事实不一致
Compose	把两个子任务组合	Planning/Agent	复杂度爆炸
Add distractor	加入无关但真实的信息	Robustness	引入歧义
Change format	JSON/table/code/diff	Structured Output	只学格式不学能力
Multi-turnify	单轮变成澄清/跟进对话	Chat/Agent	状态不一致
Environmentize	加入 repo/tool/web state	Agent	环境构造成本高

DEITA 对“复杂度”研究给出了更严格的经验：它利用 evolution 产生同源但不同复杂度的样本，再让模型对这些变体排序/打分，并训练复杂度 scorer。其 Evol Complexity 会采用添加约束、deepening、concretizing、增加 reasoning steps 等方式，实验中这类复杂度度量优于简单使用 instruction length 或 perplexity。[S7]

3.3 Generator C : Backtranslation / Source-grounded Synthesis

Instruction Backtranslation 从真实 web text 出发，让 seed model 为文档反向生成“什么用户指令会要求产生这段内容”，再自我筛选高质量样本。[S5] 这条路线很重要，因为生成锚点来自真实语料，而不是完全从生成模型的高概率想象空间中采样。

Magocoder 的 OSS-Instruct 把这个思想用于代码：从开源代码 snippet 出发，让模型构造与 snippet 相关的 coding instruction，生成 75K synthetic instruction data；论文明确把 open-source references 看作缓解纯 LLM synthetic bias、提高真实感与可控性的手段。[S6]

工程模式 Document/Repo/DB/Tool Schema → Prompt Generator → Grounded Prompt → Verifier。对于企业领域、代码仓库、Agent 工具任务，这是比“纯聊天模型凭空造题”更可靠的路线。

3.4 Generator D : Blank-template / Latent User Distribution

Magpie 的关键观察是：对一个已经对齐的 chat model，只提供对话模板到 user message 的位置，模型就可能自然续写出“用户会问什么”。它用该方法生成 4M instructions，并从中选出 300K 高质量实例。[S10]

这相当于不显式给 seed instruction，而是从 aligned model 内部已经形成的 user-query 分布中抽样。优点是摆脱人工 seed 预定义范围；风险是把 generator model 的 latent alignment distribution 当成真实世界分布。

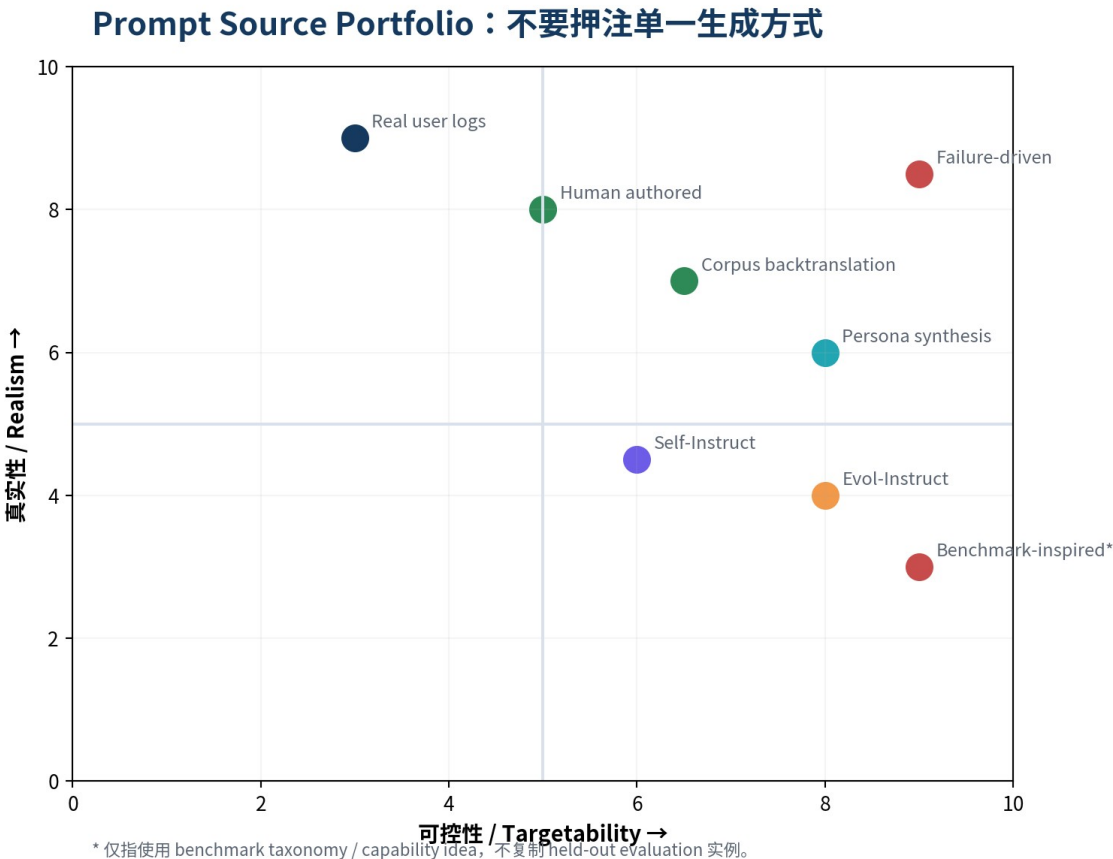
3.5 Generator E : Persona × Skill

Persona Hub 提出 persona-driven synthesis，并构建 10 亿 persona 作为多样视角载体，可用于生成数学/逻辑问题、instructions、知识文本、工具等。[S11] Tulu 3 进一步把 persona-conditioned synthesis 用在精确 instruction following、math、coding 等目标技能，并公开报告其 Prompt Pool 中约 43% 为内部 synthetic、57% 来自公开来源，总计 939,344 prompts。[S12]

```
recipe = {
  "skill": "coding",
  "persona": "ML infra engineer maintaining distributed training jobs",
  "difficulty": "hard",
  "interaction": "single-turn",
  "constraints": ["must require debugging", "must include a realistic failure symptom"],
  "grounding": "optional repo/code context",
  "output": "prompt_only"
}
```

Persona 的正确用法 Persona 不是在 prompt 前面机械添加“你是一名 X”。它真正应该改变任务分布：知识背景、目标、约束、术语、容错标准、工作流和真实使用场景。

4. Prompt Source Portfolio：真实性与可控性的权衡



一个成熟 Prompt Factory 不应该押注单一数据源。真实用户日志真实性高但可控性弱；synthetic 目标性强但可能同质；source-grounded 兼顾现实知识与控制；failure-driven 对能力缺口最精确，但容易过拟合当前 Eval。

来源	真实性	可控性	规模	最适合	主要治理
Human authored	高	高	低	高价值 anchors / safety / hard tasks	成本、一致性
Real-user logs	最高	低	高	真实分布与长尾	隐私、许可、噪声、风险
Self-/Seed synthetic	中	中高	很高	快速扩覆盖	同质化、复制 bias
Persona synthetic	中	高	很高	长尾视角、skill gap	persona 真有效性
Source-grounded	高	高	高	领域/代码/文档	来源许可、可验证性
Failure-driven	高（对目标）	最高	中	必要性、补短板	eval overfit
Adversarial/red-team	中	最高	中	边界与鲁棒性	分布偏置

4.1 Real-user Mining：为什么不能只靠 synthetic？

WildChat 公开收集了 1M 用户-ChatGPT conversations、超过 2.5M interaction turns，并报告其 user prompts 在语言和真实使用类型上具有很高多样性。[S9] 对 Prompt Factory 而言，真实日志的价值不是直接全部训练，而是提供“自然分布的校准集”：真实用户如何措辞、会省略哪些上下文、哪些任务会多轮追问、哪些请求很怪但高频。

- 先做 consent/licensing/privacy 过滤，再做语义任务标签；不要把日志清洗当成普通文本清洗。
- 对高频简单任务可下采样；对长尾高价值任务可上采样，避免自然流量决定训练优先级。
- 真实日志可以作为“prompt realism evaluator”的参考分布，而不仅是训练数据本身。

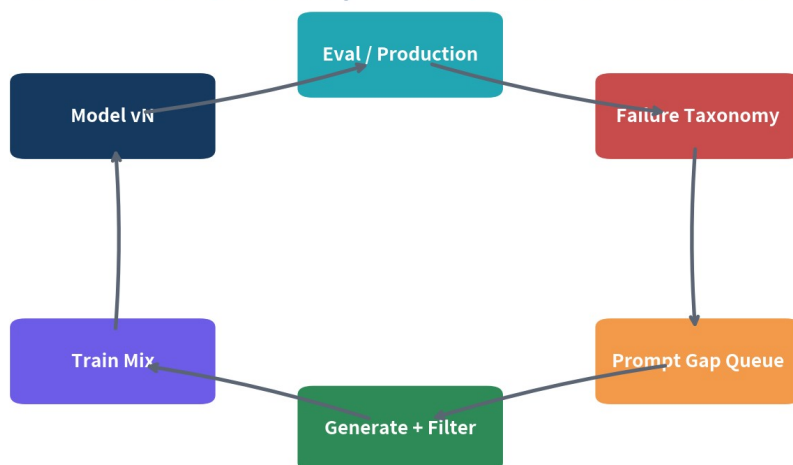
4.2 多生成：UltraChat 出的示

UltraChat 不依赖真实用户查询，而是系统化生成约 1.5M multi-turn dialogues，目标是覆盖人类与 AI 助手可能发生的广泛交互。[S4] 它说明 Prompt Factory 不应该永远输出“单个 user prompt”；对聊天与 Agent，输出单元可能是 dialogue skeleton：用户初始目标 + follow-up + correction + constraints drift。

4.3 Failure-driven：从“可能有用”到“当前必要”

Ai2 在 DR Tulu 的 cold-start 数据中，会从 SearchArena/OpenScholar 等真实交互来源筛选真正需要多步搜索、规划与综合的 prompt，并混入短链可验证任务和额外 challenging synthetic prompts，以保持长短任务覆盖。[S14] 这是一种更接近产品闭环的 Prompt Factory：先发现真实能力缺口，再生成对应训练经验。

Failure-driven Prompt Factory：Prompt Pool 是动态控制面，不是一次性数据文件



5. Prompt Contract：Prompt 不应只是一个 text 字段

如果 Prompt Factory 的输出只有 {"prompt": "..."}，后续几乎无法解释数据分布、调试失败、重采样、做 contamination audit 或跨训练阶段路由。建议把 Prompt 定义为版本化训练资产。

```

{
  "prompt_id": "pf_2026_000123",
  "text": "...",
  "task": {
    "capability": "coding.debugging",
    "domain": "python.backend",
    "difficulty": 4,
    "interaction": "agentic",
    "language": "zh-CN"
  },
  "source": {
    "type": "persona_synthetic",
    "seed_id": "seed_042",
    "generator": "model_x",
    "recipe_version": "evol-v3"
  }
}
  
```

```
{,
  "constraints": ["modify existing code", "run tests"],
  "grounding": {"repo_snapshot": "...", "tool_schema": "..."},
  "verification": {"type": "unit_tests", "available": true},
  "scores": {"quality": 0.93, "novelty": 0.81, "difficulty": 0.76},
  "dedup_cluster": "c_1842",
  "contamination": {"eval_overlap": false},
  "stage_eligibility": ["sft", "dpo", "rlvr"]
}
```

字段组	必须回答的问题	为什么重要
Task metadata	这条 Prompt 训练什么能力？	连接 Task Distribution / Eval
Source lineage	它从哪里来、谁生成、由哪个 seed 演化？	追踪 bias 与许可
Difficulty	当前模型是否处于可学习区间？	Curriculum / hard mining
Constraints	需要满足哪些显式条件？	Instruction-following 评测
Grounding	是否依赖文档、repo、tool state？	可复现与 verifier
Scores	质量/新颖/必要性等如何？	选择与重采样
Dedup cluster	与哪些 prompt 语义同簇？	控制重复
Contamination	是否接近 eval/test？	防止虚假提升
Stage eligibility	适合 SFT/DPO/RL 哪一阶段？	不同训练信号需要不同 prompt

6. Prompt Quality：先判断“问题值不值得训练”

Prompt Quality Funnel：生成量只是入口，训练价值来自过滤后的“有效信号”



注：宽度为概念示意，不代表固定保留率。

Prompt Quality 与 Response Quality 要分开建模。Prompt 的高质量至少包含：task validity、clarity、answerability、target fit、difficulty fit、novelty、realism、groundability/verifiability、policy/safety compatibility。

维度	高质量定义	典型坏例	自动检查
Validity	确实定义了可执行任务	自相矛盾/缺主语	LLM rubric + rules
Clarity	信息足以理解要求	核心条件模糊	ambiguity scorer
Answerability	在给定上下文下可完成	缺关键数据	grounding completeness
Target fit	命中目标 capability bucket	表面 coding 实为常识问答	task classifier
Difficulty fit	落在目标难度区间	太易/不可解	difficulty scorer / model success rate
Novelty	提供新训练信号	同义改写泛滥	embedding / intent dedup
Realism	像真实用户/任务	人为模板痕迹重	realism judge / log comparison
Verifiability	结果可客观核验	开放写作硬塞 binary reward	verifier availability
Cost fit	训练/rollout 成本合理	超长但无新增能力	token/tool budget

6.1 DEITA：复杂度、质量、多样性三维选择

DEITA 在大数据池上系统研究 complexity、quality、diversity，并以 score-first + diversity-aware 的方式选数据；仅使用 6K SFT 样本即可达到或超过多种更大数据集基线。[S7] 对 Prompt Factory 的启示是：生成端必须同时产出可计算的“数据属性”，否则只能按来源或随机抽样。

DEITA 的实验还表明，instruction length 并不是好的复杂度指标，perplexity 也可能表现很差；而 evolution-based complexity 通过同源多阶段变体建立更细粒度排序信号。[S7] 所以难度控制不能偷懒用“更长 = 更难”。

6.2 MoDS / LESS : 加入“必要性”和“目标影响”

MoDS 的思路是从质量、coverage、necessity 三方面选择 instruction data，其中“necessity”来自当前模型在哪些样本上做得差；这把 prompt selection 从静态质量问题推进到 model-relative 问题。[S8a]

LESS 则通过 optimizer-aware 的低秩梯度相似度估计数据对目标能力的影响，在实验中选择 5% 数据常能超过使用全量数据。[S8] 这提示 Prompt Factory 的最终目标不是最大化“通用好看分数”，而是最大化在目标模型、目标能力、当前训练阶段下的边际训练价值。

推荐的统一 Utility
$$U(p) = w_{\text{gap}} \cdot \text{Gap}(p) + w_{\text{quality}} \cdot Q(p) + w_{\text{novel}} \cdot \text{Novelty}(p) + w_{\text{fit}} \cdot \text{DifficultyFit}(p) + w_{\text{verify}} \cdot \text{Verifiability}(p) - w_{\text{cost}} \cdot \text{Cost}(p) - w_{\text{risk}} \cdot \text{Risk}(p) - w_{\text{dup}} \cdot \text{Redundancy}(p)$$

7. Diversity : 去重不是为了“数据漂亮”，而是为了边际学习价值

Synthetic Prompt 的常见失败是“句子看起来不一样，但训练信号高度重复”。因此 Diversity 至少要分 8 个层次，而不能只用编辑距离。

层次	问题	可用信号
Lexical	词面是否重复？	n-gram / MinHash / ROUGE
Semantic	语义是否近重复？	embedding nearest neighbor
Intent	目标动作是否相同？	intent/task tags
Domain	主题/专业领域是否覆盖？	domain classifier
Persona	用户背景与目标是否覆盖？	persona tags
Format	表格/JSON/code/自然语言等是否多样？	format parser
Interaction	单轮、多轮、澄清、纠错、tool 等是否覆盖？	conversation state tags
Difficulty	是否集中在一个难度？	difficulty bins / success-rate bins

Self-Instruct 使用 ROUGE-L < 0.7 作为 instruction novelty gate，是早期实用方案。[S1] DEITA 进一步采用 embedding 表征距离来维持 diversity，并在受控实验中发现 diversity-aware selection 优于随机选择。[S7] 对大规模工厂，更合理的是层级去重：exact → lexical → embedding → task/intent cluster。

关于 synthetic data 的后续研究也发现，数据 diversity 与训练效果存在正相关，尤其会影响后续 supervised fine-tuning 的收益。[S15] 因此“生成更多”通常不如“增加新的有效簇”。

7.1 Coverage Map 与 Empty Cell 生成

推荐把 Prompt Pool 映射到高维 Task Cube，然后找空桶与低覆盖桶，而不是盲目随机生成。

```
coverage_key = (
    capability,
```

```
difficulty_bin,
domain,
interaction,
language,
context_length_bin,
verifier_type
)

for bucket in target_distribution:
    deficit = target_count[bucket] - current_high_quality_count[bucket]
    if deficit > 0:
        generation_queue.push(bucket, priority=deficit * eval_gap[bucket])
```

8. Difficulty Factory：让 Prompt 落在模型“可学习边界”

难度的工程目标不是越高越好，而是让大多数训练样本落在 current model 的“learnable frontier”。太简单没有梯度，太难则产生随机失败、错误 response 和高方差 reward。

Difficulty zone	当前模型表现	训练价值	Prompt Factory 策略
Too Easy	几乎全对	低	下采样/保留少量行为锚点
Learnable	稳定但仍有错误	最高	主训练区间
Frontier	部分能解	高	多 rollout + verifier + curriculum
Too Hard	几乎全错	低/危险	分解任务或降低复杂度
Adversarial	特意攻击弱点	评测/鲁棒性高	单独 bucket，限制比例

Prompt difficulty 可以由“结构特征 + 模型相对表现”共同估计：约束数、需要的 reasoning steps、工具链长度、上下文长度、信息整合跨度、成功率、best-of-N success 等。对于 RLVR，模型在同一 prompt 上既不是 0% 也不是 100% 成功，通常更有训练信息。

8.1 Difficulty Mutation 的验收顺序

- 7. 生成原始 prompt p0；
- 8. 选择单一 mutation operator（如 add constraint）；
- 9. 生成 p1，并验证语义目标仍一致；
- 10. 检查 answerability / contradiction；
- 11. 用 target model 或 proxy model 测试 success-rate 变化；
- 12. 若难度没有上升，继续演化；若变成不可解，回退；
- 13. 把 (p0,p1,operator,score_delta) 写入 lineage，供后续学习 difficulty scorer。

9. Grounding 与 Verifiability：高价值 Prompt 的两个加速器

在 coding、math、tool、SQL、retrieval、agent 等任务中，Prompt Factory 能否同时构造 verifier，常常决定后续 RL/selection 能否规模化。一个 prompt 若无法可靠判断是否成功，其大规模 rollout 价值会显著下降。

任务类型	Prompt grounding	Verifier	Prompt Factory 应保存
Math	题目条件/数值	标准答案 / symbolic check	answer schema
Code	repo snapshot / tests	unit tests / compiler	commit hash / test cmd
SQL	DB schema / data snapshot	query result	DB version
Tool use	tool schema / environment	task state	tool version / initial state
Retrieval	document corpus	evidence/answer match	doc ids / snapshot
Agent	sandbox world state	goal state / task success	environment seed

这也是 source-grounded synthesis（Instruction Backtranslation、OSS-Instruct）重要的原因：它们为 prompt 提供了外部锚点，减少“凭空生成的问题”。[S5][S6]

10. Contamination：Prompt Factory 必须内建 Eval 防火墙

Prompt generation 极易把 benchmark 变成训练数据，尤其当 generator 接触过公开 benchmark、工程人员直接用 benchmark 样本改写、或用“生成和某 benchmark 类似的问题”却没有隔离具体测试实例。结果是评测分数上升但泛化没有提升。

风险源	错误做法	正确做法
直接改写 held-out item	换数字/换人名后训练	只使用能力 taxonomy，不暴露 test item
公共数据重叠	混入已有 benchmark train/test	hash/substring/embedding decontamination
Generator memorization	生成近似公开题	对候选与 eval corpus 做相似度扫描
Failure-driven overfit	只围绕同一 eval 模板造题	抽象 failure mechanism，再跨域/跨格式生成
多阶段泄漏	DPO/RL prompt 命中 final eval	所有 stage 共用 contamination registry

Tülu 3 公开强调数据 provenance 与 decontamination，并剔除与其 evaluation suite 重叠超过阈值的数据集；其整体 recipe 把 decontamination 当成 Prompt/Data curation 的基本步骤。[S12]

11. Stage Router：SFT、DPO、RLVR 不应该吃同一批 Prompt

Stage	需要什么 Prompt	为什么	典型来源
SFT	代表理想行为、覆盖核心技能	建立 behavior prior	真实/高质 synthetic/source-grounded

DPO	能产生有区分度的多个 response	需要 chosen vs rejected margin	新 prompt + SFT 部分复用
RLVR	可自动验证、难度处于 frontier	需要可靠 reward 与探索空间	math/code/tool/verifiable tasks
General RL	环境能返回任务成功信号	优化 multi-step policy	agent/sandbox tasks
Distillation	teacher 能显著优于 student	压缩强行为	hard prompts / teacher search

Tulu 3 的公开实验给出一个很实用的线索：在 preference tuning 中，增加 unique prompts 能提高下游表现；在 DPO mix 中引入“未在 SFT 使用的新 prompts”也能带来收益；on-policy completions 同样重要。[S12] 这意味着 Prompt Factory 应维护 stage-level novelty，而不是把 SFT prompt pool 机械复用到底。

11.1 Prompt × Completion 解耦

资产	核心指标	主要生成器	主要验收
Prompt	覆盖/真实/难度/必要性/可验证	seed/evol/persona/log/corpus	task validity + diversity + target fit
Completion	正确/帮助/风格/安全	teacher/on-policy models	verifier/judge
Preference Pair	margin/错误类型/代表性	multi-model sampling	pair quality / judge consistency
Trajectory	task success/step efficiency/recovery	agent rollout	environment state + outcome

工程收益 同一高价值 Prompt 可以在不同阶段多次“再完成”：SFT 用最强 teacher completion；DPO 用多模型候选；RL 用当前 policy on-policy rollout。Prompt 资产的生命周期因此比单次 instruction-response pair 更长。

12. Generator Portfolio：不要让一个模型垄断 Prompt 分布

Generator model 自身就是数据分布的一部分。用单一强模型大量合成，容易得到统一的语气、任务模板、价值偏好和知识边界。Ensemble-Instruct 的研究也探索了使用异构模型和 ensemble 来提高 synthetic instruction quality。[S16a]

维度	策略	Know-Why
Model family	多家族/多 checkpoint	降低单一模型 inductive bias
Aligned vs Base	混合 aligned elicitation 与 base generation	aligned 更会造“用户问题”，base 可能提供不同分布
Temperature	按任务类型配置，而非全局固定	创意任务需多样，严格格式需稳定
System prompt	多 recipe 版本 A/B	防止模板单一
Seed source	human + real + synthetic 混合	避免 synthetic feedback loop
Response model	与 prompt generator 分离	降低两端同源错误相关性

12.1 如何避免 Synthetic Feedback Loop

- 保留真实/人工 anchor set，持续测量 synthetic 与 real prompt distribution divergence。
- 定期更换 generator / recipe；不要只把上一轮 synthetic prompt 回灌为下一轮 seed。
- 使用 source-grounded / persona / real-user 三类外部锚点增加新信息。
- 按 cluster 的“新增覆盖”而不是 raw count 作为生成 KPI。
- 将 generator ID 写入 lineage，能在模型出现异常风格时追溯来源。

13. Coding / Agent Prompt Factory：从“问题文本”升级为“任务环境”

对 Coding Agent，真正的 prompt 往往不是一句自然语言，而是一个可复现环境任务：repo snapshot + issue + tools + tests + permissions + initial state。Prompt Factory 需要与 sandbox/environment factory 联动。

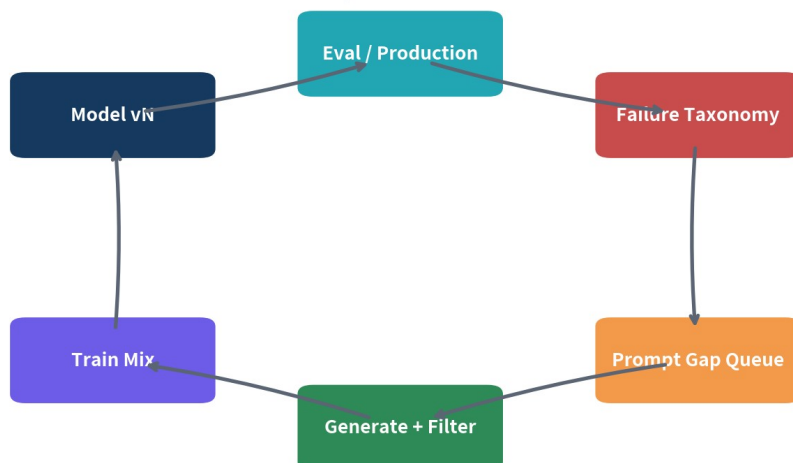
Bucket	Prompt 需要构造	Verifier / Outcome	生成策略
Repo understanding	多文件依赖 + 查询目标	定位/回答正确	真实 repo + source-grounded
Bug fixing	失败症状 + 隐含 root cause	tests pass	历史 bug / mutation
Editing	已有代码 + 明确变更约束	diff/test	real patch backtranslation
Tool use	工具 schema + 需要检索的信息	正确 tool/args	tool-aware synthesis
Recovery	预置一次 tool/test failure	最终完成且恢复	environment perturbation
Long horizon	多个依赖子目标	task complete	compose + curriculum
Constraint following	禁止改某文件/限制 API	policy + tests	add-constraint evolution

Magocoder 的 OSS-Instruct 可视为 coding prompt factory 的早期 source-grounded 范式：用真实开源代码 snippet 约束生成空间，减少纯 synthetic bias。[S6] 对 repo-level agent，可以把 snippet 升级为 commit / issue / test failure / dependency graph。

```
coding_prompt_asset = {
  "issue": "...",
  "repo_commit": "abc123",
  "visible_files": ["src/..."],
  "hidden_tests": ["..."],
  "tool_policy": ["read", "grep", "edit", "test"],
  "initial_failures": ["pytest::test_x"],
  "success": "all target tests pass and no forbidden files changed"
}
```

14. Failure-driven Prompt Factory：动态重写训练分布

Failure-driven Prompt Factory：Prompt Pool 是动态控制面，不是一次性数据文件



最先进的 Prompt Factory 不再以“本季度生成 5M prompts”为目标，而以“缩小目标能力 gap”为目标。每轮 Eval/Production failure 都进入 Failure Taxonomy，形成 Gap Queue；Prompt Factory 只对高优先级 gap 分配生成预算。

Failure signal	抽象 failure mechanism	不要做	应该生成
Tool 选错	tool disambiguation 弱	复制原失败请求	跨领域的相似 tool-choice 边界题
代码 patch 过大	locality/control 弱	只换文件名	不同 repo/语言的最小修改任务
多约束漏一条	constraint tracking 弱	机械增加更多字	不同格式、不同约束类型的组合
长文检索漏证据	evidence coverage 弱	复制原文档	不同文档结构的 multi-hop evidence tasks
测试失败后停止	recovery policy 弱	重复同一 bug	预置不同 failure mode 的 recovery tasks

关键原则 Failure-driven synthesis 的单位应是“failure mechanism”，不是“failure example”。这样才能补能力而不是背题。

14.1 Gap Queue 优先级

```
priority(bucket) =  
    product_impact(bucket)  
    * eval_gap(bucket)  
    * learning_feasibility(bucket)  
    * verifier_reliability(bucket)  
    / generation_and_training_cost(bucket)
```

再乘一个 novelty / saturation 修正：

```
priority *= max(0, 1 - coverage_saturation(bucket))
```

15. Prompt Factory KPI：不要用“生成条数”当北极星

KPI	定义	坏指标替代品
Valid Yield	通过 task-valid/answerable 筛选的比例	raw generation count
Unique Cluster Yield	每 1K 生成带来的新语义/任务簇数量	去重后条数
Coverage Gap	目标 Task Cube 未覆盖比例	数据总量
Frontier Rate	落在可学习难度区间的比例	平均 prompt 长度
Verifier Coverage	具备可靠 verifier 的 prompt 比例	是否“看起来可测”
Realism Score	与真实用户/ workflow 分布的一致性	LLM 单一打分
Necessary Rate	当前模型实际失败/低置信的样本比例	“难题”标签
Downstream Lift / 1K	每千条 prompt 带来的 eval 提升	训练 loss
Stage Novelty	DPO/RL 相对 SFT 的 prompt 新颖度	阶段数据规模

15.1 Prompt Factory 的实验最小集

- 14. Source ablation：real vs synthetic vs persona vs grounded；
- 15. Seed ablation：不同 seed coverage 与数量；
- 16. Diversity ablation：raw vs lexical dedup vs embedding/task-aware dedup；
- 17. Difficulty ablation：easy / frontier / hard mixture；
- 18. Generator ablation：单模型 vs multi-generator；
- 19. Stage novelty ablation：DPO/RL reuse-only vs mixed new prompts；
- 20. Failure-driven ablation：static pool vs gap-aware dynamic sampling；
- 21. Cost-normalized comparison：同样 token/GPU/label budget 下比较 downstream lift。

16. 生产级 Prompt Factory：服务与数据层拆解

服务/组件	职责	典型接口
Taxonomy Service	能力树/难度/域标签	classify(prompt) → bucket
Seed Registry	人工/真实/synthetic seed 版本管理	sample(bucket, policy)
Generation Orchestrator	批量调度 generator/recipe	generate(recipe, quota)
Mutation Engine	evol/rephrase/compose	mutate(prompt, operator)
Grounding Builder	文档/repo/tool/env 绑定	build_context(asset)
Quality Scorer	validity/clarity/answerability	score(prompt)
Difficulty Scorer	绝对 + model-relative 难度	difficulty(prompt, model)
Dedup/Cluster Service	多层近重复与簇管理	nearest(prompt)
Contamination Scanner	eval/test 交叉检查	scan(prompt, eval_registry)
Prompt Registry	lineage/version/metadata	register(asset)
Stage Router	SFT/DPO/RL/Distill 路由	eligible(prompt, stage)
Gap Controller	根据 eval failure 重加权	update_distribution(eval)

16.1 推的据生命周期

DRAFT
→ GENERATED
→ TASK_VALID
→ ANSWERABLE
→ DEDUPED
→ CONTAMINATION_CLEAN
→ SCORED
→ APPROVED
→ STAGE_ASSIGNED
→ USED_IN_RUN_<id>
→ EVAL_ATTRIBUTED
→ RETIRED / REWEIGHTED / MUTATED

17. Prompt Factory 成熟度模型：L0 → L4

等级	形态	能力	主要瓶颈
L0 • Data Dump	人工拼接公开 prompt	能训练	不可解释、重复、污染
L1 • Generator	单模型批量 synthetic	能扩量	同质/质量不稳
L2 • Curated Factory	taxonomy + scoring + dedup + lineage	可控覆盖	仍是静态生成
L3 • Multi-source Factory	real + persona + grounded + multi-generator + stage routing	真实与目标兼顾	反馈还不自动
L4 • Adaptive Controller	eval/failure → gap queue → dynamic generation/reweight	持续补短板	需要成熟 eval/verifier/infra

判断标准 如果团队仍然通过“某数据集占 30%、某 synthetic 文件占 20%”手工维护 Prompt，而无法回答某条 prompt 为什么存在、对应哪个能力 gap、被哪次训练使用、是否带来 downstream lift，那么还没有真正进入 Prompt Factory 阶段。

18. 可直接复用的 Prompt Factory Cookbook

18.1 第一步：建立 Prompt Distribution Contract

```
prompt_distribution:
  coding.debugging:
    weight: 0.18
    difficulty: {easy: 0.1, learnable: 0.45, frontier: 0.35, hard: 0.1}
    source_mix: {real: 0.25, grounded: 0.45, persona: 0.20, failure: 0.10}
    verifier_required: true
  instruction_following.precise:
    weight: 0.12
    source_mix: {human: 0.15, persona: 0.55, evol: 0.30}
    constraints_range: [2, 7]
```

18.2 第二步：为每个 bucket 选择 generation recipe

Bucket	主 Recipe	辅 Recipe	为什么
General chat	real-user + Magpie-like	persona	保持自然分布
Precise IF	persona × constraints	Evol	可精确控制约束
Math reasoning	source/seed + difficulty mutation	persona	可验证 + 多样背景
Coding	repo/code-grounded	Evol / failure	真实性 + tests
Tool use	tool-schema grounded	failure-driven	参数/选择可测
Agent	environment/task template	compose + recovery mutation	长程轨迹

18.3 第三步：两段式验收

Gate A: Prompt QA	Gate B: Training Utility
validity	target gap
clarity	novelty
answerability	difficulty fit
grounding completeness	verifier availability
contamination clean	cost-normalized utility

18.4 第四步：Completion Factory 接管

只有通过 Prompt QA 的资产才进入 response generation。针对同一 prompt，可按训练阶段生成不同 completion：SFT 用强 teacher；DPO 采多候选并构造 chosen/rejected；RL 使用当前 policy on-policy rollout。

18.5 第五步：训练后做 attribution

每次训练记录 prompt bucket、source、generator、recipe、difficulty、stage 与 downstream eval。长期积累后，可以估计“哪类 prompt 对哪类能力有效”，再反向更新 Prompt Factory 的生成策略。

19. Prompt Factory 的 15 类常见失败模式

失败模式	症状	根因	修复
1. Seed lock-in	生成内容围绕少数模板	seed 覆盖窄	taxonomy-first seeds + 多 source
2. Verb diversity illusion	动词不同但任务相同	只做词面统计	intent/embedding clustering
3. Length = difficulty	prompt 越来越长	错误代理指标	model-relative difficulty
4. Impossible evolution	复杂题自相矛盾	mutation 无 verifier	answerability gate
5. Persona cosplay	只是加角色前缀	persona 不改变任务	让 persona 改变目标/约束/知识
6. Same-model coupling	同模型造题+答案	错误相关	prompt/completion decouple
7. Synthetic monoculture	统一语气/结构	单 generator	multi-generator + real anchors
8. Real-log domination	高频闲聊占满数据	自然流量 ≠ 训练价值	target distribution reweight
9. Benchmark leakage	eval 暴涨	test prompt 改写进训练	eval firewall
10. Unverifiable RL prompts	reward 噪声高	问题无法自动判断	verifier-first design
11. Static prompt pool	模型进步后仍训旧难度	无反馈控制	failure-driven refresh
12. Stage reuse saturation	DPO/RL 全复用 SFT	缺少新探索输入	stage novelty quotas
13. Dedup overkill	删除真正不同任务	只看 embedding	semantic + task metadata combined
14. Dedup underkill	海量近义改写	阈值太松	cluster-level quotas
15. No lineage	无法知道数据来源	只存最终 JSONL	registry + version + recipe trace

20. 公开研究中的 Prompt Factory 关键实证锚点

研究	Prompt/Data 构造亮点	对工程的意义
Self-Instruct [S1]	175 seeds; 52,445 instructions; ROUGE-L novelty filter; 人工抽检仅 54% 完整样本全有效	bootstrapping + Prompt/Completion 解耦
Unnatural Instructions [S2]	3-shot 生成 64K，再 rephrase 到约 240K	生成新任务与表述扩增可拆分
WizardLM [S3]	Evol-Instruct 逐步提高 instruction complexity	difficulty 可成为显式生成控制量
UltraChat [S4]	约 1.5M synthetic multi-turn dialogues	Prompt 单元可升级为对话轨迹
Instruction Backtranslation [S5]	从 human-written web text 反推 instruction	用真实语料 grounding 生成
Magocoder [S6]	OSS code snippets → 75K coding	代码任务应由 repo/source 锚定

	instructions	
DEITA [S7]	complexity + quality + diversity; 6K 高质量样本可很强	数据属性与选择比 raw scale 更关键
LESS [S8]	5% influence-selected data 常超过 full data	prompt utility 应与目标能力相关
WildChat [S9]	1M conversations, >2.5M turns	真实分布是 synthetic 的校准锚点
Magpie [S10]	blank chat template → 4M instructions → 300K selected	可从 aligned model latent user distribution 抽样
Persona Hub [S11]	10 亿 personas	persona 可系统化扩大长尾视角
Tulu 3 [S12]	939,344 prompts; 57% public + 43% synthetic; persona 填 skill gaps	现代 Hybrid Prompt Portfolio 案例

21. 最终检查清单：上线一个 Prompt Factory 前必须回答

- ☐ 是否先有 Objective Contract 与 Task Distribution，而不是先生成数据？
- ☐ 是否有清晰 task taxonomy 与 bucket quota？
- ☐ 每条 prompt 是否有 source/seed/generator/recipe lineage？
- ☐ 是否区分 Prompt Quality 与 Completion Quality？
- ☐ 是否有至少两类外部锚点（真实用户、语料、repo、persona 等）防止 synthetic monoculture？
- ☐ 是否用 model-relative signal 控制 difficulty，而不是用长度？
- ☐ 是否做 exact + lexical + semantic + intent/task 多层去重？
- ☐ 是否有 eval contamination registry 与 stage-wide scanner？
- ☐ 是否能为 RL/Agent prompt 绑定可靠 verifier/environment？
- ☐ 是否为 SFT/DPO/RL 设置不同 prompt eligibility 与 novelty quota？
- ☐ 是否把 Eval/production failure 自动变成 Gap Queue？
- ☐ 是否记录 prompt bucket 对 downstream lift 的归因，支持下一轮 reweight？

一句话总结 Prompt Factory 的真正目标不是“制造问题”，而是持续制造“对当前模型而言有边际学习价值、与目标能力一致、可验证且不污染评测的训练经验入口”。

参考文献与公开技术资料

ID	作者/机构	资料	出处
[S1]	Wang et al.	Self-Instruct: Aligning Language Models with Self-Generated Instructions	ACL 2023 / arXiv:2212.10560
[S2]	Honovich et al.	Unnatural Instructions: Tuning Language Models with (Almost) No Human Labor	arXiv:2212.09689
[S3]	Xu et al.	WizardLM: Empowering Large Language Models to Follow Complex Instructions	ICLR 2024 / arXiv:2304.12244
[S4]	Ding et al.	Enhancing Chat Language Models by Scaling High-quality Instructional Conversations (UltraChat)	arXiv:2305.14233
[S5]	Li et al.	Self-Alignment with Instruction Backtranslation	arXiv:2308.06259
[S6]	Wei et al.	Magocoder: Empowering Code Generation with OSS-Instruct	arXiv:2312.02120
[S7]	Liu et al.	What Makes Good Data for Alignment? / DEITA	ICLR 2024 / arXiv:2312.15685
[S8]	Xia et al.	LESS: Selecting Influential Data for Targeted Instruction Tuning	ICML 2024 / arXiv:2402.04333
[S8a]	Du et al.	MoDS: Model-oriented Data Selection for Instruction Tuning	arXiv:2311.15653
[S9]	Zhao et al.	WildChat: 1M ChatGPT Interaction Logs in the Wild	arXiv:2405.01470
[S10]	Xu et al.	Magpie: Alignment Data Synthesis from Scratch by Prompting Aligned LLMs with Nothing	arXiv:2406.08464
[S11]	Ge et al.	Scaling Synthetic Data Creation with 1,000,000,000 Personas	arXiv:2406.20094 (v3, 2025)
[S12]	Lambert et al. / Ai2	Tulu 3: Pushing Frontiers in Open Language Model Post-Training + official technical blog	arXiv:2411.15124; Ai2, 2024
[S13]	Qwen Team	Qwen2.5 Technical Report	arXiv:2412.15115
[S14]	Ai2	DR Tulu: An open, end-to-end training recipe for long-form deep research	Ai2 technical blog, 2025
[S15]	Chen et al.	On the Diversity of Synthetic Data and its Impact on Training Large Language Models	arXiv:2410.15226
[S16a]	Lee et al.	Ensemble-Instruct: Generating Instruction-Tuning Data with a Heterogeneous Mixture of LMs	arXiv:2310.13961
[S16]	Grattafiori et al.	The Llama 3 Herd of Models	arXiv:2407.21783

研究说明：本文把“Prompt Factory”作为工程抽象进行综合，不声称上述论文都使用这一术语。文中的架构、Utility 公式、成熟度模型、Prompt Contract、Failure-driven 控制环等属于基于公开研究证据的工程综合与推演；具体数值事实均按相应来源标注。